

Riffi News Ingestion Engine – Build Brief

Oct 4, 2026 · @Zyra

Why we are building this

We need an engine that pulls every new development on our tracked topics, every day, from about 130 sources, so the team and the seed-account agent always have fresh hooks for opinion content.

Riffi launches in Bengaluru at the end of October or the first week of November 2026. The plan needs 1,000+ content pieces on day one and 50–100 seed accounts posting opinion takes on fresh news, fed by an AI agent reading RSS feeds. Riffi is for opinions, not news: we never republish articles. We need the trigger that starts a debate, as early as possible.

The engine has two jobs:

1. **Daily intake:** fetch, clean, de-duplicate, tag by topic and score for relevance, then hand a ranked list to the content team and the post-generation agent.
2. **Source proof:** within two weeks, show which sources actually deliver relevant updates and which are dead weight, so we keep, fix or drop each one on evidence.

The KPI is **update recall**. Whenever something real happens on a tracked topic, at least one source should surface it within 24 hours. A topic with two updates a month that catches both scores 100%. Days with no news don't count against anything.

Audience and what counts as relevant

Relevant means: it's about a tracked topic, it's a new development, and our audience would argue about it.

Audience: aspirational, English-speaking tier-1 crowd in Bengaluru earning ₹1–2L+ a month. Corporate and startup professionals, tech workers, founders, and tier-1 college students (Christ University and similar). Heavy X and Instagram users. Not tier-2/3 mass audiences.

Label	What it looks like	Example
High	New development on a High-priority topic, Bengaluru or Karnataka, clear debate angle	GBA poll notification issued; Pink Line opening date announced; CEO says 70-hour week again
Medium	New development on a tracked topic with a weaker local angle	RBI rate decision; a pan-India film release date shifts
Low	On-topic but no new development, or no debate angle	Explainer recapping old facts; routine traffic advisory
Drop	Off-topic, duplicate, stale, or excluded	Crime blotter; horoscope; communal flashpoint; article older than 7 days

An item older than 72 hours at first fetch is stale unless it is the first time any source carried it.

Subject matter

We track 151 topics across four geographies, weighted towards Bengaluru. All of them are in `topics.csv` with an ID, category, priority and debate angles.

Geography	Weight	What we want
Bengaluru	Highest	City life and civic pain: traffic, metro, rent, potholes, floods, water, GBA polls, pubs, RCB, neighbourhoods, food, dating, startup and work culture
Karnataka	High	State politics (CM, cabinet, opposition), guarantees and Shakti, caste survey, Kannada language and signage, Kannada cinema, Bigg Boss Kannada, holidays and festivals
South India	Medium	Delimitation and tax devolution, Mekedatu and Cauvery, language policy, South cinema

Geography	Weight	What we want
Pan-India	Medium, higher with a Bengaluru angle	Work culture (70-hour week, WFH, AI and jobs), H-1B, tax and personal finance, F&O, national politics without communal framing, Bollywood vs South, cricket, big national events

Three topic types (all in `topics.csv`):

- **Dated events (D01–D36)**: fixed-date events between 25 Oct and 7 Dec 2026. We need any change to the date, venue or status, plus the news when it happens. Source: [launch calendar](#).
- **Ongoing topics (O01–O28)**: live stories such as GBA polls, the tunnel road, CJP, the Darshan trial. We need every new development.
- **Evergreen topics (B, K, SI, P IDs, 87 in all)**: perennial debates such as rent, traffic and city wars. We need the news that makes one of them flare up again. Source: [evergreen library](#).

Excluded entirely: communal or religious flashpoints. That includes Tipu Jayanti, conversion bills, cattle bills, "infiltrators" rows, the Dharmasthala case, temple-fund bills and hijab or halal rows. Items on these are dropped, not ranked low.

Sensitive but allowed: Kannada vs Hindi, locals vs migrants, reservation and the caste survey, the local job quota, delimitation, stray dogs. Tag them with a `sensitive` flag so a human reviews the framing before anything is posted.

The sources

The engine reads 131 active sources from `feeds.csv` (an export of [Source List v4](#)). Each has a `route_type` that decides how it is fetched.

Route type	Sources	How the engine fetches it	Watch out for
Google News RSS	60	Plain RSS GET on <code>news.google.com/rss/search?q=...</code>	Links are Google redirects; titles end in " - Publisher"; resolve to the real URL

Route type	Sources	How the engine fetches it	Watch out for
X / Instagram via RSS.app	25	Skip in phase 1. Fetch the <code>backup_google_news_url</code> instead. Only the 5 rows marked <code>priority_x_feed</code> get a paid RSS.app feed later	X blocks scrapers; RSS.app is paid and breaks
Web page monitor	22	Fetch the page, extract main text, hash it, and emit an item when the hash changes (or run <code>changedetection.io</code> and read its RSS)	Government sites are slow, use PDFs and may block bots
Native publisher RSS/Atom	18	Plain RSS or Atom GET	Some redirect (Inc42 /feed to /feed/); some need a browser user-agent
RSSHub Telegram	3	GET <code>rsshub.app/telegram/channel/<name></code> ; self-host RSSHub if rate-limited	Post text sits in the description; no separate headline
YouTube Atom	2	GET <code>youtube.com/feeds/videos.xml?channel_id=...</code>	Prajavani needs its channel ID looked up
Manual	1	Not fetched (traffic police ASTraM app)	Listed for completeness

Only 2 feeds are confirmed working so far: RBI notifications and Inc42. Every other feed is untested, so the first run is also the first real test.

`blocklist.csv` lists copycat and unreliable domains. Drop any item whose resolved URL matches one of them.

What the engine must do

The engine runs on a schedule, turns every source into clean items, and produces one ranked daily list plus a health report.

1. **Load sources** from `feeds.csv` (later from the admin panel's database). Skip blocklisted and Manual rows.
2. **Fetch** each source on its schedule:

- Every 30 min: Google News queries for High-priority topics, Telegram, priority X feeds.
 - Every 2 hours: all other RSS, Atom and Google News feeds.
 - Every 6 hours: web page monitors (government and ticketing sites).
 - Browser-like user-agent, 20 s timeout, 2 retries with backoff, conditional GET (ETag / Last-Modified) where supported.
3. **Parse** RSS, Atom and HTML into one item shape (see Data model). Keep the raw payload for debugging.
 4. **Resolve** Google News redirect links to the real article URL and publisher. Strip " - Publisher" from titles.
 5. **De-duplicate**: same resolved URL = same item. Near-identical normalised titles within 48 hours = same story. Group them into one story cluster that lists every source that carried it.
 6. **Filter**: drop blocklisted domains, items older than 7 days, and items that match an excluded topic.
 7. **Tag** each story with one or more topic IDs from `topics.csv` (see Tagging).
 8. **Score** relevance 0–100 and give it a label: High, Medium, Low or Drop.
 9. **Store** sources, fetch runs, items, story clusters and tags in a database.
 10. **Output** every day at 7:00 IST:
 - A ranked digest of the last 24 hours (High first), as a dashboard page plus a CSV or Google Sheet export.
 - A feed health report (see Health checks).
 - A JSON endpoint or table the seed-account agent can read.

Phase 1 runs as a standalone service with a CLI. If the Riffi admin panel already exists in the repo, the engine plugs into its database and UI instead of building new ones.

Data model

Five tables cover everything: sources, fetch runs, items, story clusters and topic tags.

Table	Key fields
<code>sources</code>	source_id, name, category, route_type, fetch_url, backup_google_news_url, tier, priority_x_feed, active, last_ok_at, consecutive_failures

Table	Key fields
fetch_runs	run_id, source_id, started_at, http_status, duration_ms, items_returned, newest_item_at, fields_present, error
items	item_id, source_id, title, url (resolved), publisher, published_at, fetched_at, summary, image_url, author, language, raw_guid, raw_payload, cluster_id
story_clusters	cluster_id, headline, first_seen_at, source_count, sources, relevance_score, label, sensitive
item_topics	cluster_id, topic_id, match_method (keyword / llm), confidence

Where each field comes from:

Field	Native RSS/Atom	Google News RSS	RSSHub Telegram	YouTube Atom	Page monitor
title	title	title, minus " - Publisher"	first line of the post	title	page title + "updated"
url	link	resolve redirect	post link	video link	page URL
publisher	source name	<source> element	channel name	channel name	source name
published_at	pubDate / updated	pubDate	pubDate	published	time the change was detected
summary	description (strip HTML)	snippet (thin)	full post text	media:description	text diff of what changed
image_url	enclosure / media:content	usually none	inline image	media:thumbnail	none

Record which fields each source actually returns in `fetch_runs.fields_present` , so we know what the post-generation step can rely on per source.

Tagging and relevance scoring

Tagging is two-step: a cheap keyword pass first, then an LLM check on anything that matched or looks local.

1. **Keyword pass:** for each topic, generate 5–15 keywords and aliases from its title and debate angles (e.g. the tunnel road topic: "tunnel road", "Hebbal", "Silk Board", "B-SMILE", "Adani"). Store them in an editable `topic_keywords` file so the team can tune them. Match on title + summary, case-insensitive, English and Kannada spellings where known.
2. **LLM pass:** send each story cluster's headline and summary, plus the candidate topics, to Claude. It returns the matching topic IDs, a one-line "what's new", a debate angle, and whether the item hits an excluded topic. Batch 20 stories per call to keep cost low.
3. **Untagged but local:** stories mentioning Bengaluru or Karnataka that match no topic go to an "unmatched local" bucket. A human reviews it weekly to find new topics.

Relevance score (0–100):

Factor	Points
Topic priority (from <code>topics.csv</code>): High / Medium / Low	30 / 20 / 10
Geography: Bengaluru / Karnataka / South India / Pan-India	25 / 20 / 12 / 10
New development (LLM says it's new, not a recap)	15
Debate angle present (LLM)	10
Source spread: 3+ sources in the cluster / 2 / 1	10 / 5 / 0
Source tier: Official / Media / Aggregator	10 / 6 / 3

Labels: 70+ High, 45–69 Medium, 20–44 Low, under 20 Drop. Excluded topics are Drop regardless of score. Sensitive topics keep their score but carry a `sensitive` flag for human review.

The weights are a starting point. Make them a config file so we can tune them after the two-week test.

Feed health checks and the daily report

Every fetch is logged, and any source failing 3 runs in a row is flagged for replacement.

Check	Recorded	Fails when
Reachable	HTTP status, response time	403, 404, 5xx or timeout
Valid	Parsed as RSS, Atom or HTML	Wrong format or broken XML
Alive	Items returned, newest item date	0 items, or newest item older than 7 days (30 days for page monitors)
Fields	Which fields are present	Missing title, link or date
Useful	Items tagged to a topic, items labelled High or Medium	0 tagged items in 7 days

Daily report (7:00 IST), one page:

- Top stories of the last 24 hours, ranked, with topics, sources and a debate angle each.
- Topics with new developments today, and High-priority topics that have had nothing for 7+ days.
- Source health: working, failing, stale, and flagged for replacement.
- Counts: items fetched, clusters, High / Medium / Low / Drop.

Write four columns back to the source list so the sheet stays current: `last_status`, `last_ok_at`, `newest_item_at`, `fields_present`.

Two-week test and success metrics

After 14 days of running, we keep, fix or drop each source based on measured recall, not on guesses.

Ground truth: each day an editor spends 10 minutes logging the real developments on High-priority topics, from any source (X, WhatsApp, TV, friends). The engine provides a simple form or sheet for this: date, topic ID, what happened, where they saw it. The engine then checks whether it caught each one and how many hours late.

Metric	Target
Update recall (ground-truth developments the engine caught)	95% across High-priority topics
Timeliness (caught within 24 hours)	90%

Metric	Target
Precision (items labelled High that the editor agrees are relevant)	70%
Working feeds (passing health checks)	90% of non-X sources

Decision rules at day 14:

- Drop a source with 0 unique catches (nothing that another source didn't already bring).
- Fix or replace a source failing health checks 3+ days in a row.
- Add a source wherever the editor's log shows the same topic being missed twice.
- Pay for RSS.app only for an X handle that broke news first at least twice.

Good live test events in the window: MLC poll counting (27 Oct), the Pink Line opening, Kannada Rajyotsava (1 Nov), CJP protest legs, the GBA poll notification and Bengaluru Tech Summit (17-19 Nov).

Claude Code prompt

Put `feeds.csv`, `topics.csv`, `blocklist.csv` and this doc (exported as Markdown, saved as `BRIEF.md`) in the repo root, then paste the prompt below into Claude Code.

You are building the news ingestion engine for Riffi, an opinion-first social platform launching in Bengaluru (end Oct / early Nov 2026). Read BRIEF.md in full before writing any code. It explains why we need this, who the audience is, what counts as relevant, and every requirement. This prompt is the build checklist.

INPUT FILES (repo root)

- `feeds.csv`: 131 active sources. Columns: `source_id`, `name`, `category`, `route_type`, `fetch_url`, `link_or_handle`, `topics_hint`, `tier`, `priority_x_feed`, `backup_google_news_url`, `known_status`, `notes`.
- `topics.csv`: 151 topics we track. Columns: `topic_id` (D = dated event, 0 = ongoing, B/K/SI/P = evergreen), `type`, `topic`, `date_or_window`, `category`, `geography`, `status`, `why_people_talk`, `debate_angles`, `spice`, `noise`, `priority`, `sensitive_note`.
- `blocklist.csv`: copycat and unreliable domains. Drop any item from these.
- `BRIEF.md`: the full spec.

STEP 0: LOOK BEFORE BUILDING

- Check whether this repo already has the Riffi admin panel (look for an existing backend, database schema, ORM models, or a feeds/sources table). If it does, integrate with its stack, database and UI. Do not build a parallel system.
- If there is no admin panel, build a standalone Python 3.11 service: SQLite for phase 1 (easy to swap for Postgres), feedparser + httpx for fetching, trafilatura for page text, APScheduler for scheduling, Typer for the CLI, and a minimal FastAPI dashboard.
- Tell me which path you chose and why before writing code, then proceed.

STEP 1: FETCHERS (one per route_type)

- "Native publisher RSS/Atom" and "Google News RSS": GET fetch_url and parse with feedparser.
- "X/Instagram via RSS.app": do NOT scrape X or Instagram. Fetch backup_google_news_url instead and mark the source as running on its backup. Leave a stub for RSS.app feed URLs to be added later, only for rows where priority_x_feed = yes.
- "Web page monitor": GET the page, extract main text with trafilatura, hash it, store the snapshot, and emit one item (title = page title + " updated", summary = a short text diff) when the hash changes. Ignore whitespace and date-stamp-only changes.
- "RSSHub Telegram": GET the rsshub.app URL; make the RSSHub base URL a config value so we can self-host.
- "YouTube Atom": parse as Atom; skip rows whose fetch_url is not a real URL (e.g. "Needs channel ID") and list them in the report.
- "Manual": skip.
- All fetchers: browser-like User-Agent, 20 s timeout, 2 retries with exponential backoff, conditional GET (ETag / Last-Modified), and per-domain rate limiting (max 1 request per 2 s per domain). Never crash the run on one bad source; log the error and continue.

STEP 2: NORMALISE AND CLEAN

- Map every entry into the items shape in BRIEF.md (title, url, publisher, published_at in IST, summary, image_url, author, language, raw_guid, raw_payload).
- Google News: resolve the news.google.com redirect to the real article URL (follow redirects; if that fails, decode the article URL from the link). Take publisher from the <source> element and strip " - Publisher" from the title.
- Strip HTML from summaries. Detect language (English or Kannada).
- Drop items whose resolved domain is in blocklist.csv, and items older than 7 days.

STEP 3: DE-DUPLICATE INTO STORY CLUSTERS

- Same resolved URL = same item.
- Normalised titles with high similarity (e.g. rapidfuzz token_set_ratio >= 85) published within 48 hours = same story cluster. Keep every source in the cluster; source_count drives scoring.

STEP 4: TAG TOPICS

- Generate a topic_keywords.yaml from topics.csv: 5 to 15 keywords and aliases per topic from its title, debate_angles and why_people_talk, including common Kannada-English spellings (Bengaluru/Bangalore, Rajyotsava, etc.). Write it to disk so a human can edit it, and reload it on every run.
- Keyword pass on title + summary.
- LLM pass with the Anthropic API (model and API key from env vars), batching 20 clusters per call. For each cluster return JSON: topic_ids, is_new_development (bool), whats_new (one line), debate_angle (one line), excluded (bool), excluded_reason.
- Excluded topics (communal or religious flashpoints: Tipu Jayanti, conversion bills, cattle bills, infiltrators rows, Dharmasthala case, temple-fund bills, hijab/halal rows) are labelled Drop.
- Bengaluru/Karnataka stories that match no topic go to an unmatched_local bucket.
- Topics with a sensitive_note get sensitive = true on their clusters.

STEP 5: SCORE

- Implement the relevance score table in BRIEF.md exactly, with all weights in scoring.yaml.
- Labels: 70+ High, 45-69 Medium, 20-44 Low, under 20 Drop.

STEP 6: STORE

- Tables: sources, fetch_runs, items, story_clusters, item_topics, page_snapshots, ground_truth. Seed sources from feeds.csv and topics from topics.csv. Make re-importing either CSV idempotent.

STEP 7: SCHEDULE

- Every 30 min: Google News feeds tied to High-priority topics, RSSHub Telegram feeds.
- Every 2 h: all other RSS/Atom/Google News feeds.
- Every 6 h: web page monitors.
- Daily at 07:00 IST: build the digest and health report.
- Also expose CLI commands: `fetch --all`, `fetch --source S004`, `test-feeds`, `digest --date`, `report`, `import-sources`, `import-topics`.

STEP 8: OUTPUTS

- Dashboard pages: (1) today's ranked stories with topics, sources, whats_new and debate_angle, filterable by label, category, geography and topic; (2) source health; (3) topic coverage: last development per topic, with High-priority topics silent for 7+ days highlighted; (4) unmatched_local bucket.
- Daily digest as Markdown and CSV in /reports/YYYY-MM-DD/.
- A JSON endpoint GET /api/stories?since=&label= that the seed-account agent will read.
- Export source health columns (last_status, last_ok_at, newest_item_at, fields_present) to sources_health.csv so we can paste them into the source sheet.

STEP 9: HEALTH CHECKS

- Implement the checks table in BRIEF.md. Record http_status, duration_ms, items_returned, newest_item_at and fields_present on every run.
- Flag a source for replacement after 3 consecutive failures, or 0 topic-tagged items in 7 days.

STEP 10: TWO-WEEK RECALL TEST

- Add a simple form (dashboard page plus CSV import) for the editor's ground-truth log: date, topic_id, what happened, where seen, time seen.
- Each night, match ground-truth entries to story clusters (same topic, within 48 hours, LLM-confirmed same event). Compute recall, timeliness (hours late) and per-source unique catches.
- At day 14, produce recall_report.md: recall per topic and per source, sources with 0 unique catches, sources failing health checks, and topics missed twice or more. Apply the decision rules in BRIEF.md as recommendations only; don't delete anything automatically.

FIRST RUN (do this before scheduling)

1. Run `test-feeds` across all 131 sources and print a table: source_id, name, route_type, HTTP status, items, newest item date, fields present, pass/fail.
2. Show me the failures with a suggested fix for each (wrong URL, needs user-agent, needs self-hosted RSSHub, site blocks bots).
3. Run one full fetch, tag and score cycle and show me the top 30 stories with their topics and scores, so I can check relevance before we go live.

RULES

- Never scrape X, Instagram or WhatsApp directly.
- Never publish or post anything. This engine only collects, ranks and reports.
- Keep secrets in .env; add .env.example.
- Write tests for parsing, Google News link resolution, de-duplication and

scoring.

- Add a README covering setup, env vars, CLI commands and how to add a source or topic.
- If something in BRIEF.md is ambiguous, make a sensible choice, note it in DECISIONS.md, and carry on.